



Exploring and pre-processing a dataset using Pandas

Estimated time needed: **30** minutes

Objectives

After completing this lab you will be able to:

- Explore the dataset
- Pre-process dataset as required (may be for visualization)

Introduction

The aim of this lab is to provide you a refresher on the **Pandas** library, so that you can pre-process and analyse the datasets before applying data visualization techniques on it. This lab will work as a crash course on *pandas*. If you are interested in learning more about the *pandas* library, detailed description and explanation of how to use it and how to clean, munge, and process data stored in a *pandas* dataframe are provided in other IBM courses.

Table of Contents

1. Exploring Datasets with *pandas*
2. The Dataset: Immigration to Canada from 1980 to 2013
3. *pandas* Basics
4. *pandas* Intermediate: Indexing and Selection
5. *pandas* Filtering based on a criteria
6. *pandas* Sorting Values

Exploring Datasets with *pandas*

pandas is an essential data analysis toolkit for Python. From their [website](#):

pandas is a Python package providing fast, flexible, and expressive data structures designed to make working with “relational” or “labeled” data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, **real world** data analysis in Python.

The course heavily relies on *pandas* for data wrangling, analysis, and visualization. We encourage you to spend some time and familiarize yourself with the *pandas* API

Reference: <http://pandas.pydata.org/pandas-docs/stable/api.html>.

The Dataset: Immigration to Canada from 1980 to 2013

Dataset Source: [International migration flows to and from selected countries - The 2015 revision](#).

The dataset contains annual data on the flows of international immigrants as recorded by the countries of destination. The data presents both inflows and outflows according to the place of birth, citizenship or place of previous / next residence both for foreigners and nationals. The current version presents data pertaining to 45 countries.

In this lab, we will focus on the Canadian immigration data.

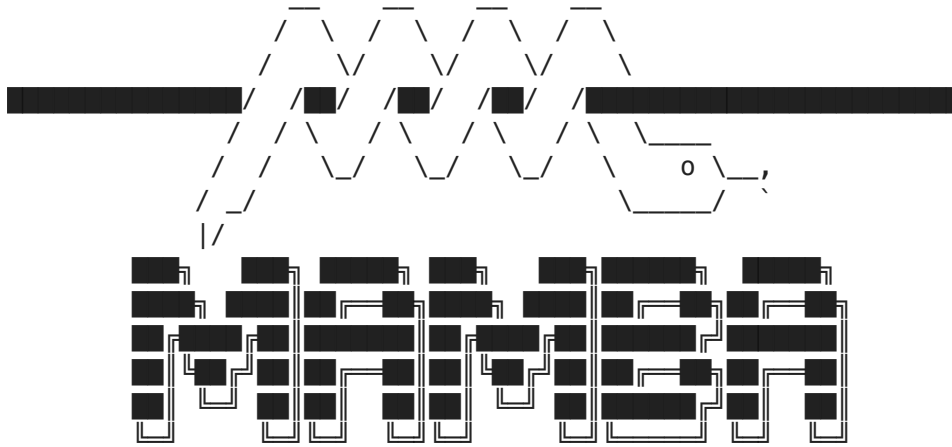
[illegible]

The Canada Immigration dataset can be fetched from [here](#).

pandas Basics

The first thing we'll do is install **openpyxl** (formerly **xlrd**), a module that *pandas* requires to read Excel files.

```
In [1]: !mamba install openpyxl==3.0.9 -y
```



mamba (1.4.2) supported by @QuantStack

GitHub: <https://github.com/mamba-org/mamba>

Twitter: <https://twitter.com/QuantStack>



Looking for: ['openpyxl==3.0.9']

```
[+] 0.0s
[+] 0.1s
pkgs/main/linux-64 0.0 B / ???.?MB @ ???.?MB/s
0.1s
pkgs/main/noarch 0.0 B / ???.?MB @ ???.?MB/s
0.1s
pkgs/r/linux-64 0.0 B / ???.?MB @ ???.?MB/s
0.1s
pkgs/r/noarch 0.0 B / ???.?MB @ ???.?MB/s
0.1s[+] 0.2s
pkgs/main/linux-64 0.0 B / ???.?MB @ ???.?MB/s
0.2s
pkgs/main/noarch 0.0 B / ???.?MB @ ???.?MB/s
0.2s
pkgs/r/linux-64 0.0 B / ???.?MB @ ???.?MB/s
0.2s
pkgs/r/noarch 0.0 B / ???.?MB @ ???.?MB/s
0.2s[+] 0.3s
pkgs/main/linux-64 303.1kB / ???.?MB @ 1.1MB/s
0.3s
pkgs/main/noarch 254.0kB / ???.?MB @ 937.9kB/s
0.3s
pkgs/r/linux-64 315.4kB / ???.?MB @ 1.1MB/s
0.3s
pkgs/r/noarch 430.0kB / ???.?MB @ 1.5MB/s
0.3s[+] 0.4s
pkgs/main/linux-64 716.8kB / ???.?MB @ 1.9MB/s
0.4s
pkgs/main/noarch 806.9kB / ???.?MB @ 2.1MB/s
0.4s
pkgs/r/linux-64 1.2MB / ???.?MB @ 3.2MB/s
```

```

0.4s
pkgs/r/noarch           1.2MB / ???.?MB @ 3.0MB/s
0.4spkgs/main/noarch          992.6kB @ 2.4MB/s
0.4s
[+] 0.5s
pkgs/main/linux-64  1.3MB @ 2.6MB/s
0.5s
pkgs/r/linux-64  1.9MB @ 3.9MB/s Finalizing
0.5s
pkgs/r/noarch  1.4MB @ 3.2MB/s
0.5spkgs/r/linux-64          @ 3.9MB/s 0.5s
[+] 0.6s
pkgs/main/linux-64  1.8MB / ???.?MB @ 3.1MB/s
0.6s
pkgs/r/noarch  2.2MB / ???.?MB @ 3.8MB/s
0.6spkgs/r/noarch          2.3MB @ 3.8MB/s
0.6s
[+] 0.7s
pkgs/main/linux-64  1.9MB / ???.?MB @ 2.9MB/s
0.7s[+] 0.8s
pkgs/main/linux-64  2.4MB / ???.?MB @ 3.1MB/s
0.8s[+] 0.9s
pkgs/main/linux-64  2.9MB / ???.?MB @ 3.3MB/s
0.9s[+] 1.0s
pkgs/main/linux-64  3.3MB / ???.?MB @ 3.4MB/s
1.0s[+] 1.1s
pkgs/main/linux-64  3.7MB / ???.?MB @ 3.4MB/s
1.1s[+] 1.2s
pkgs/main/linux-64  4.2MB / ???.?MB @ 3.6MB/s
1.2s[+] 1.3s
pkgs/main/linux-64  4.5MB / ???.?MB @ 3.5MB/s
1.3s[+] 1.4s
pkgs/main/linux-64  4.5MB / ???.?MB @ 3.5MB/s
1.4s[+] 1.5s
pkgs/main/linux-64  5.4MB / ???.?MB @ 3.6MB/s
1.5s[+] 1.6s
pkgs/main/linux-64  5.8MB / ???.?MB @ 3.6MB/s
1.6s[+] 1.7s
pkgs/main/linux-64  5.9MB / ???.?MB @ 3.6MB/s
1.7s[+] 1.8s
pkgs/main/linux-64  6.1MB / ???.?MB @ 3.5MB/s
1.8s[+] 1.9s
pkgs/main/linux-64  6.6MB / ???.?MB @ 3.5MB/s
1.9s[+] 2.0s
pkgs/main/linux-64  6.9MB / ???.?MB @ 3.5MB/s
2.0s[+] 2.1s
pkgs/main/linux-64  7.5MB / ???.?MB @ 3.6MB/s
2.1s[+] 2.2s
pkgs/main/linux-64  8.1MB / ???.?MB @ 3.7MB/s
2.2s[+] 2.3s
pkgs/main/linux-64  8.7MB / ???.?MB @ 3.8MB/s
2.3s[+] 2.4s
pkgs/main/linux-64  8.9MB / ???.?MB @ 3.8MB/s
2.4s[+] 2.5s
pkgs/main/linux-64  9.2MB / ???.?MB @ 3.8MB/s
2.5s[+] 2.6s

```

```
pkgs/main/linux-64 10.2MB / 10.2MB @ 3.9MB/s
2.6s[+] 2.7s
pkgs/main/linux-64 10.6MB / 10.6MB @ 4.0MB/s
2.7s[+] 2.8s
pkgs/main/linux-64 11.3MB / 11.3MB @ 4.1MB/s
2.8s[+] 2.9s
pkgs/main/linux-64 11.5MB @ 4.1MB/s Finalizing
2.9s[+] 3.0s
pkgs/main/linux-64 11.5MB @ 4.1MB/s Finalizing
3.0s[+] 3.1s
pkgs/main/linux-64 11.5MB @ 4.1MB/s Finalizing
3.1s[+] 3.2s
pkgs/main/linux-64 11.5MB @ 4.1MB/s Finalizing
3.2s[+] 3.3s
pkgs/main/linux-64 11.5MB @ 4.1MB/s Finalizing
3.3s[+] 3.4s
pkgs/main/linux-64 11.5MB @ 4.1MB/s Finalizing
3.4s[+] 3.5s
pkgs/main/linux-64 11.5MB @ 4.1MB/s Finalizing
3.5spkgs/main/linux-64 @ 4.1MB/s 3.5s
```

Pinned packages:

- python 3.7.*

Transaction

Prefix: /home/jupyterlab/conda/envs/python

Updating specs:

- openpyxl==3.0.9
- ca-certificates
- certifi
- openssl

Package	Version	Build	Channel	Size
Install:				
+ et_xmlfile	1.1.0	py37h06a4308_0	pkgs/main/linux-64	10kB
+ openpyxl	3.0.9	pyhd3eb1b0_0	pkgs/main/noarch	168kB
Upgrade:				
- ca-certificates	2023.5.7	hbcca054_0	conda-forge	
+ ca-certificates	2025.12.2	h06a4308_0	pkgs/main/linux-64	128kB
- openssl	1.1.1t	h0b41bf4_0	conda-forge	
+ openssl	1.1.1w	h7f8727e_0	pkgs/main/linux-64	4MB

Summary:

Install: 2 packages

Upgrade: 2 packages

Total download: 4MB

—

```

[+] 0.0s
Downloading 0.0s 0.0 B
Extracting 0.0s 0
[+] 0.1s
Downloading (4) 0.0s 0.0 B ca-certificates
Extracting 0.0s 0
[+] 0.2s
Downloading (4) 0.1s 0.0 B ca-certificates
Extracting 0.0s 0
0.0sca-certificates 128.4kB @ 501.9kB/s
0.3s
et_xmlfile 10.2kB @ 39.8kB/s 0.3s
openpyxl 167.6kB @ 649.8kB/s 0.3s
[+] 0.3s
Downloading (1) 0.2s 448.8kB openssl
Extracting (3) 0.0s 0 ca-certificates
[+] 0.4s
Downloading 0.3s 4.2MB
Extracting (4) 0.1s 0 ca-certificates
0.1sopenssl 3.9MB @ 9.9MB/s
0.4s
[+] 0.5s
Downloading 0.3s 4.2MB
Extracting (4) 0.2s 0 ca-certificates
[+] 0.6s
Downloading 0.3s 4.2MB
Extracting (4) 0.3s 0 ca-certificates
[+] 0.7s
Downloading 0.3s 4.2MB
Extracting (4) 0.4s 0 et_xmlfile
[+] 0.8s
Downloading 0.3s 4.2MB
Extracting (4) 0.5s 0 et_xmlfile
[+] 0.9s
Downloading 0.3s 4.2MB

```

```

Extracting (4) -  0 et_xmlfile
0.6s[+] 1.0s
Downloading  4.2MB
0.3s
Extracting (4)  0 et_xmlfile
0.7s[+] 1.1s
Downloading  4.2MB
0.3s
Extracting (4)  0 openpyxl
0.8s[+] 1.2s
Downloading  4.2MB
0.3s
Extracting (4)  0 openpyxl
0.9s[+] 1.3s
Downloading  4.2MB
0.3s
Extracting (4)  0 openpyxl
1.0s[+] 1.4s
Downloading  4.2MB
0.3s
Extracting (4)  0 openpyxl
1.1s[+] 1.5s
Downloading  4.2MB
0.3s
Extracting (4)  0 openssl
1.2s[+] 1.6s
Downloading  4.2MB
0.3s
Extracting (3)  1 openssl
1.3s[+] 1.7s
Downloading  4.2MB
0.3s
Extracting (3)  1 openssl
1.4s[+] 1.8s
Downloading  4.2MB
0.3s
Extracting (3)  1 openssl
1.5s[+] 1.9s
Downloading  4.2MB
0.3s
Extracting (3)  1 et_xmlfile
1.6s[+] 2.0s
Downloading  4.2MB
0.3s
Extracting (1)  3 openssl
1.7s[+] 2.1s
Downloading  4.2MB
0.3s
Extracting  4
1.8s
Downloading and Extracting Packages

```

```

Preparing transaction: done
Verifying transaction: done
Executing transaction: done

```

Next, we'll do is import two key data analysis modules: *pandas* and *numpy*.

```
In [3]: import numpy as np # useful for many scientific computing in Python
import pandas as pd # primary data structure library
```

Let's download and import our primary Canadian Immigration dataset using *pandas*'s `read_excel()` method.

```
In [4]: df_can = pd.read_excel(
    'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDE
    sheet_name='Canada by Citizenship',
    skiprows=range(20),
    skipfooter=2)

print('Data read into a pandas dataframe!')
```

Data read into a pandas dataframe!

Let's view the top 5 rows of the dataset using the `head()` function.

```
In [5]: df_can.head()
# tip: You can specify the number of rows you'd like to see as follows: df_c
```

```
Out[5]:
```

	Type	Coverage	OdName	AREA	AreaName	REG	RegName	DEV	DevN
0	Immigrants	Foreigners	Afghanistan	935	Asia	5501	Southern Asia	902	Develc req
1	Immigrants	Foreigners	Albania	908	Europe	925	Southern Europe	901	Develc req
2	Immigrants	Foreigners	Algeria	903	Africa	912	Northern Africa	902	Develc req
3	Immigrants	Foreigners	American Samoa	909	Oceania	957	Polynesia	902	Develc req
4	Immigrants	Foreigners	Andorra	908	Europe	925	Southern Europe	901	Develc req

5 rows × 43 columns

We can also view the bottom 5 rows of the dataset using the `tail()` function.

```
In [6]: df_can.tail()
```

Out [6]:

	Type	Coverage	OdName	AREA	AreaName	REG	RegName	DEV	DevName
190	Immigrants	Foreigners	Viet Nam	935	Asia	920	South-Eastern Asia	902	Developing region
191	Immigrants	Foreigners	Western Sahara	903	Africa	912	Northern Africa	902	Developing region
192	Immigrants	Foreigners	Yemen	935	Asia	922	Western Asia	902	Developing region
193	Immigrants	Foreigners	Zambia	903	Africa	910	Eastern Africa	902	Developing region
194	Immigrants	Foreigners	Zimbabwe	903	Africa	910	Eastern Africa	902	Developing region

5 rows × 43 columns

When analyzing a dataset, it's always a good idea to start by getting basic information about your dataframe. We can do this by using the `info()` method.

This method can be used to get a short summary of the dataframe.

In [7]: `df_can.info(verbose=False)`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 195 entries, 0 to 194
Columns: 43 entries, Type to 2013
dtypes: int64(37), object(6)
memory usage: 65.6+ KB
```

To get the list of column headers we can call upon the data frame's `columns` instance variable.

In [8]: `df_can.columns`

```
Out[8]: Index([      'Type', 'Coverage', 'OdName', 'AREA', 'AreaName', 'REG',
      'RegName', 'DEV', 'DevName', 1980, 1981, 1982, 1983, 1984, 1985, 1986, 1987, 1988, 1989, 1990, 1991, 1992, 1993, 1994, 1995, 1996, 1997, 1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013],
      dtype='object')
```

Similarly, to get the list of indices we use the `.index` instance variables.

```
In [10]: df_can.index
```

```
Out[10]: RangeIndex(start=0, stop=195, step=1)
```

Note: The default type of instance variables `index` and `columns` are **NOT** `list`.

```
In [11]: print(type(df_can.columns))  
print(type(df_can.index))
```

```
<class 'pandas.core.indexes.base.Index'>
```

```
<class 'pandas.core.indexes.range.RangeIndex'>
```

To get the index and columns as lists, we can use the `tolist()` method.

```
In [12]: df_can.columns.tolist()
```

```
Out [12]: ['Type',  
          'Coverage',  
          'OdName',  
          'AREA',  
          'AreaName',  
          'REG',  
          'RegName',  
          'DEV',  
          'DevName',  
          1980,  
          1981,  
          1982,  
          1983,  
          1984,  
          1985,  
          1986,  
          1987,  
          1988,  
          1989,  
          1990,  
          1991,  
          1992,  
          1993,  
          1994,  
          1995,  
          1996,  
          1997,  
          1998,  
          1999,  
          2000,  
          2001,  
          2002,  
          2003,  
          2004,  
          2005,  
          2006,  
          2007,  
          2008,  
          2009,  
          2010,  
          2011,  
          2012,  
          2013]
```

```
In [13]: df_can.index.tolist()
```

```
Out[13]: [0,  
          1,  
          2,  
          3,  
          4,  
          5,  
          6,  
          7,  
          8,  
          9,  
          10,  
          11,  
          12,  
          13,  
          14,  
          15,  
          16,  
          17,  
          18,  
          19,  
          20,  
          21,  
          22,  
          23,  
          24,  
          25,  
          26,  
          27,  
          28,  
          29,  
          30,  
          31,  
          32,  
          33,  
          34,  
          35,  
          36,  
          37,  
          38,  
          39,  
          40,  
          41,  
          42,  
          43,  
          44,  
          45,  
          46,  
          47,  
          48,  
          49,  
          50,  
          51,  
          52,  
          53,  
          54,  
          55,
```

56,
57,
58,
59,
60,
61,
62,
63,
64,
65,
66,
67,
68,
69,
70,
71,
72,
73,
74,
75,
76,
77,
78,
79,
80,
81,
82,
83,
84,
85,
86,
87,
88,
89,
90,
91,
92,
93,
94,
95,
96,
97,
98,
99,
100,
101,
102,
103,
104,
105,
106,
107,
108,
109,
110,
111,

112,
113,
114,
115,
116,
117,
118,
119,
120,
121,
122,
123,
124,
125,
126,
127,
128,
129,
130,
131,
132,
133,
134,
135,
136,
137,
138,
139,
140,
141,
142,
143,
144,
145,
146,
147,
148,
149,
150,
151,
152,
153,
154,
155,
156,
157,
158,
159,
160,
161,
162,
163,
164,
165,
166,
167,

```

168,
169,
170,
171,
172,
173,
174,
175,
176,
177,
178,
179,
180,
181,
182,
183,
184,
185,
186,
187,
188,
189,
190,
191,
192,
193,
194]

```

```

In [14]: print(type(df_can.columns.tolist()))
         print(type(df_can.index.tolist()))

```

```

<class 'list'>
<class 'list'>

```

To view the dimensions of the dataframe, we use the `shape` instance variable of it.

```

In [15]: # size of dataframe (rows, columns)
         df_can.shape

```

```

Out[15]: (195, 43)

```

Note: The main types stored in *pandas* objects are `float`, `int`, `bool`, `datetime64[ns]`, `datetime64[ns, tz]`, `timedelta[ns]`, `category`, and `object` (string). In addition, these dtypes have item sizes, e.g. `int64` and `int32`.

Let's clean the data set to remove a few unnecessary columns. We can use *pandas* `drop()` method as follows:

```

In [16]: # in pandas axis=0 represents rows (default) and axis=1 represents columns.
         df_can.drop(['AREA', 'REG', 'DEV', 'Type', 'Coverage'], axis=1, inplace=True)
         df_can.head(2)

```

```
Out[16]:
```

	OdName	AreaName	RegName	DevName	1980	1981	1982	1983	1984	1985	1986	1987	1988	1989	1990	1991	1992	1993	1994	1995	1996	1997	1998	1999	2000	2001	2002	2003	2004	2005	2006	2007	2008	2009	2010	2011	2012	2013
0	Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	34	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	
1	Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

2 rows x 38 columns

Let's rename the columns so that they make sense. We can use `rename()` method by passing in a dictionary of old and new names as follows:

```
In [17]: df_can.rename(columns={'OdName':'Country', 'AreaName':'Continent', 'RegName':  
df_can.columns
```

```
Out[17]: Index([ 'Country', 'Continent', 'Region', 'DevName', 1980,  
1981, 1982, 1983, 1984, 1985,  
1986, 1987, 1988, 1989, 1990,  
1991, 1992, 1993, 1994, 1995,  
1996, 1997, 1998, 1999, 2000,  
2001, 2002, 2003, 2004, 2005,  
2006, 2007, 2008, 2009, 2010,  
2011, 2012, 2013],  
dtype='object')
```

We will also add a 'Total' column that sums up the total immigrants by country over the entire period 1980 - 2013, as follows:

```
In [18]: df_can['Total'] = df_can.sum(axis=1)  
df_can['Total']
```

```
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages/ipykernel_launcher.py:1: FutureWarning: Dropping of nuisance columns in DataFrame reductions (with 'numeric_only=None') is deprecated; in a future version this will raise TypeError. Select only valid columns before calling the reduction.  
"""Entry point for launching an IPython kernel.
```

```
Out[18]: 0      58639  
1      15699  
2      69439  
3         6  
4         15  
...  
190     97146  
191         2  
192     2985  
193     1677  
194     8598  
Name: Total, Length: 195, dtype: int64
```

We can check to see how many null objects we have in the dataset as follows:

```
In [19]: df_can.isnull().sum()
```

```
Out[19]: Country      0
          Continent    0
          Region       0
          DevName       0
          1980          0
          1981          0
          1982          0
          1983          0
          1984          0
          1985          0
          1986          0
          1987          0
          1988          0
          1989          0
          1990          0
          1991          0
          1992          0
          1993          0
          1994          0
          1995          0
          1996          0
          1997          0
          1998          0
          1999          0
          2000          0
          2001          0
          2002          0
          2003          0
          2004          0
          2005          0
          2006          0
          2007          0
          2008          0
          2009          0
          2010          0
          2011          0
          2012          0
          2013          0
          Total         0
          dtype: int64
```

Finally, let's view a quick summary of each column in our dataframe using the `describe()` method.

```
In [20]: df_can.describe()
```

Out [20]:

	1980	1981	1982	1983	1984	
count	195.000000	195.000000	195.000000	195.000000	195.000000	195
mean	508.394872	566.989744	534.723077	387.435897	376.497436	354
std	1949.588546	2152.643752	1866.997511	1204.333597	1198.246371	1079
min	0.000000	0.000000	0.000000	0.000000	0.000000	0
25%	0.000000	0.000000	0.000000	0.000000	0.000000	0
50%	13.000000	10.000000	11.000000	12.000000	13.000000	11
75%	251.500000	295.500000	275.000000	173.000000	181.000000	191
max	22045.000000	24796.000000	20620.000000	10015.000000	10170.000000	9564

8 rows × 35 columns

pandas Intermediate: Indexing and Selection (slicing)

Select Column

There are two ways to filter on a column name:

Method 1: Quick and easy, but only works if the column name does NOT have spaces or special characters.

```
df.column_name # returns series
```

Method 2: More robust, and can filter on multiple columns.

```
df['column'] # returns series
df[['column 1', 'column 2']] # returns dataframe
```

Example: Let's try filtering on the list of countries ('Country').

```
In [21]: df_can.Country # returns a series
```

```
Out [21]: 0      Afghanistan
1      Albania
2      Algeria
3      American Samoa
4      Andorra
...
190     Viet Nam
191  Western Sahara
192      Yemen
193      Zambia
194      Zimbabwe
Name: Country, Length: 195, dtype: object
```

Let's try filtering on the list of countries ('Country') and the data for years: 1980 - 1985.

```
In [22]: df_can[['Country', 1980, 1981, 1982, 1983, 1984, 1985]] # returns a dataframe
# notice that 'Country' is string, and the years are integers.
# for the sake of consistency, we will convert all column names to string labels
```

```
Out [22]:
```

	Country	1980	1981	1982	1983	1984	1985
0	Afghanistan	16	39	39	47	71	340
1	Albania	1	0	0	0	0	0
2	Algeria	80	67	71	69	63	44
3	American Samoa	0	1	0	0	0	0
4	Andorra	0	0	0	0	0	0
...	...	...	...	...	...	...	...
190	Viet Nam	1191	1829	2162	3404	7583	5907
191	Western Sahara	0	0	0	0	0	0
192	Yemen	1	2	1	6	0	18
193	Zambia	11	17	11	7	16	9
194	Zimbabwe	72	114	102	44	32	29

195 rows × 7 columns

Select Row

There are main 2 ways to select rows:

```
df.loc[label] # filters by the labels of the index/column
df.iloc[index] # filters by the positions of the index/column
```

Before we proceed, notice that the default index of the dataset is a numeric range from 0 to 194. This makes it very difficult to do a query by a specific country. For example to search for data on Japan, we need to know the corresponding index value.

This can be fixed very easily by setting the 'Country' column as the index using `set_index()` method.

```
In [27]: df_can.set_index('Country', inplace=True)
# tip: The opposite of set is reset. So to reset the index, we can use df_ca
```

```
In [28]: df_can.head(3)
```

```
Out[28]:
```

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985
Country									
Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340
Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0
Algeria	Africa	Northern Africa	Developing regions	80	67	71	69	63	44

3 rows × 38 columns

```
In [30]: # optional: to remove the name of the index
df_can.index.name = None
df_can.head()
```

```
Out[30]:
```

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985
Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340
Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0
Algeria	Africa	Northern Africa	Developing regions	80	67	71	69	63	44
American Samoa	Oceania	Polynesia	Developing regions	0	1	0	0	0	0
Andorra	Europe	Southern Europe	Developed regions	0	0	0	0	0	0

5 rows × 38 columns

Example: Let's view the number of immigrants from Japan (row 87) for the following scenarios: 1. The full row data (all columns) 2. For year 2013 3. For years 1980 to 1985

```
In [31]: # 1. the full row data (all columns)
df_can.loc['Japan']
```

```
Out[31]:
```

Continent	Asia
Region	Eastern Asia
DevName	Developed regions
1980	701
1981	756
1982	598
1983	309
1984	246
1985	198
1986	248
1987	422
1988	324
1989	494
1990	379
1991	506
1992	605
1993	907
1994	956
1995	826
1996	994
1997	924
1998	897
1999	1083
2000	1010
2001	1092
2002	806
2003	817
2004	973
2005	1067
2006	1212
2007	1250
2008	1284
2009	1194
2010	1168
2011	1265
2012	1214
2013	982
Total	27707

Name: Japan, dtype: object

```
In [32]: # alternate methods
df_can.iloc[87]
```

```
Out[32]:
```

Continent	Asia
Region	Eastern Asia
DevName	Developed regions
1980	701
1981	756
1982	598
1983	309
1984	246
1985	198
1986	248
1987	422
1988	324
1989	494
1990	379
1991	506
1992	605
1993	907
1994	956
1995	826
1996	994
1997	924
1998	897
1999	1083
2000	1010
2001	1092
2002	806
2003	817
2004	973
2005	1067
2006	1212
2007	1250
2008	1284
2009	1194
2010	1168
2011	1265
2012	1214
2013	982
Total	27707

Name: Japan, dtype: object

```
In [33]: df_can[df_can.index == 'Japan']
```

```
Out[33]:
```

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	..
Japan	Asia	Eastern Asia	Developed regions	701	756	598	309	246	198	248	.

1 rows x 38 columns

```
In [34]: # 2. for year 2013
df_can.loc['Japan', 2013]
```

```
Out[34]: 982
```

```
In [35]: # alternate method
# year 2013 is the last column, with a positional index of 36
df_can.iloc[87, 36]
```

Out[35]: 982

```
In [36]: # 3. for years 1980 to 1985
df_can.loc['Japan', [1980, 1981, 1982, 1983, 1984, 1984]]
```

```
Out[36]: 1980    701
1981    756
1982    598
1983    309
1984    246
1984    246
Name: Japan, dtype: object
```

```
In [37]: # Alternative Method
df_can.iloc[87, [3, 4, 5, 6, 7, 8]]
```

```
Out[37]: 1980    701
1981    756
1982    598
1983    309
1984    246
1985    198
Name: Japan, dtype: object
```

Exercise: Let's view the number of immigrants from **Haiti** for the following scenarios:

1. The full row data (all columns)
2. For year 2000
3. For years 1990 to 1995

```
In [39]: df_can.loc['Haiti']
```

```
Out[39]:
```

Continent	Latin America and the Caribbean
Region	Caribbean
DevName	Developing regions
1980	1666
1981	3692
1982	3498
1983	2860
1984	1418
1985	1321
1986	1753
1987	2132
1988	1829
1989	2377
1990	2379
1991	2829
1992	2399
1993	3655
1994	2100
1995	2014
1996	1955
1997	1645
1998	1295
1999	1439
2000	1631
2001	2433
2002	2174
2003	1930
2004	1652
2005	1682
2006	1619
2007	1598
2008	2491
2009	2080
2010	4744
2011	6503
2012	5868
2013	4152
Total	84813

Name: Haiti, dtype: object

```
In [40]: df_can.loc['Haiti', 2000]
```

```
Out[40]: 1631
```

```
In [41]: df_can.loc['Haiti', [1990, 1991, 1992, 1993, 1994, 1995]]
```

```
Out[41]:
```

1990	2379
1991	2829
1992	2399
1993	3655
1994	2100
1995	2014

Name: Haiti, dtype: object

► [Click here for a sample python solution](#)

Column names that are integers (such as the years) might introduce some confusion. For example, when we are referencing the year 2013, one might confuse that when the 2013th positional index.

To avoid this ambiguity, let's convert the column names into strings: '1980' to '2013'.

[illegible]

Since we converted the years to string, let's declare a variable that will allow us to easily call upon the full range of years:

```
# useful for plotting later on
years = list(map(str, range(1980, 2014)))
years
```

```
Out [43]: ['1980',
          '1981',
          '1982',
          '1983',
          '1984',
          '1985',
          '1986',
          '1987',
          '1988',
          '1989',
          '1990',
          '1991',
          '1992',
          '1993',
          '1994',
          '1995',
          '1996',
          '1997',
          '1998',
          '1999',
          '2000',
          '2001',
          '2002',
          '2003',
          '2004',
          '2005',
          '2006',
          '2007',
          '2008',
          '2009',
          '2010',
          '2011',
          '2012',
          '2013']
```

Exercise: Create a list named 'year' using map function for years ranging from 1990 to 2013.

Then extract the data series from the dataframe df_can for Haiti using year list.

```
In [44]: year = list(map(str, range(1990, 2014)))
df_can.loc['Haiti', year]
```

```
Out [44]: 1990    2379
          1991    2829
          1992    2399
          1993    3655
          1994    2100
          1995    2014
          1996    1955
          1997    1645
          1998    1295
          1999    1439
          2000    1631
          2001    2433
          2002    2174
          2003    1930
          2004    1652
          2005    1682
          2006    1619
          2007    1598
          2008    2491
          2009    2080
          2010    4744
          2011    6503
          2012    5868
          2013    4152
Name: Haiti, dtype: object
```

► [Click here for a sample python solution](#)

Filtering based on a criteria

To filter the dataframe based on a condition, we simply pass the condition as a boolean vector.

For example, Let's filter the dataframe to show the data on Asian countries (AreaName = Asia).

```
In [45]: # 1. create the condition boolean series
condition = df_can['Continent'] == 'Asia'
print(condition)
```

```
Afghanistan      True
Albania           False
Algeria           False
American Samoa   False
Andorra           False
...
Viet Nam          True
Western Sahara    False
Yemen             True
Zambia            False
Zimbabwe          False
Name: Continent, Length: 195, dtype: bool
```

```
In [46]: # 2. pass this condition into the dataframe  
df_can[condition]
```

Out [46]:

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985
Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340
Armenia	Asia	Western Asia	Developing regions	0	0	0	0	0	0
Azerbaijan	Asia	Western Asia	Developing regions	0	0	0	0	0	0
Bahrain	Asia	Western Asia	Developing regions	0	2	1	1	1	3
Bangladesh	Asia	Southern Asia	Developing regions	83	84	86	81	98	92
Bhutan	Asia	Southern Asia	Developing regions	0	0	0	0	1	0
Brunei Darussalam	Asia	South-Eastern Asia	Developing regions	79	6	8	2	2	4
Cambodia	Asia	South-Eastern Asia	Developing regions	12	19	26	33	10	7
China	Asia	Eastern Asia	Developing regions	5123	6682	3308	1863	1527	1816
China, Hong Kong Special Administrative Region	Asia	Eastern Asia	Developing regions	0	0	0	0	0	0
China, Macao Special Administrative Region	Asia	Eastern Asia	Developing regions	0	0	0	0	0	0
Cyprus	Asia	Western Asia	Developing regions	132	128	84	46	46	43
Democratic People's Republic of Korea	Asia	Eastern Asia	Developing regions	1	1	3	1	4	3
Georgia	Asia	Western Asia	Developing regions	0	0	0	0	0	0
India	Asia	Southern Asia	Developing regions	8880	8670	8147	7338	5704	4217
Indonesia	Asia	South-Eastern Asia	Developing regions	186	178	252	115	123	100
Iran (Islamic Republic of)	Asia	Southern Asia	Developing regions	1172	1429	1822	1592	1977	1648

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985
Iraq	Asia	Western Asia	Developing regions	262	245	260	380	428	237
Israel	Asia	Western Asia	Developing regions	1403	1711	1334	541	446	680
Japan	Asia	Eastern Asia	Developed regions	701	756	598	309	246	198
Jordan	Asia	Western Asia	Developing regions	177	160	155	113	102	179
Kazakhstan	Asia	Central Asia	Developing regions	0	0	0	0	0	0
Kuwait	Asia	Western Asia	Developing regions	1	0	8	2	1	4
Kyrgyzstan	Asia	Central Asia	Developing regions	0	0	0	0	0	0
Lao People's Democratic Republic	Asia	South-Eastern Asia	Developing regions	11	6	16	16	7	17
Lebanon	Asia	Western Asia	Developing regions	1409	1119	1159	789	1253	1683
Malaysia	Asia	South-Eastern Asia	Developing regions	786	816	813	448	384	374
Maldives	Asia	Southern Asia	Developing regions	0	0	0	1	0	0
Mongolia	Asia	Eastern Asia	Developing regions	0	0	0	0	0	0
Myanmar	Asia	South-Eastern Asia	Developing regions	80	62	46	31	41	23
Nepal	Asia	Southern Asia	Developing regions	1	1	6	1	2	4
Oman	Asia	Western Asia	Developing regions	0	0	0	8	0	0
Pakistan	Asia	Southern Asia	Developing regions	978	972	1201	900	668	514
Philippines	Asia	South-Eastern Asia	Developing regions	6051	5921	5249	4562	3801	3150
Qatar	Asia	Western Asia	Developing regions	0	0	0	0	0	0
Republic of Korea	Asia	Eastern Asia	Developing regions	1011	1456	1572	1081	847	962

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985
Saudi Arabia	Asia	Western Asia	Developing regions	0	0	1	4	1	2
Singapore	Asia	South-Eastern Asia	Developing regions	241	301	337	169	128	135
Sri Lanka	Asia	Southern Asia	Developing regions	185	371	290	197	1086	845
State of Palestine	Asia	Western Asia	Developing regions	0	0	0	0	0	0
Syrian Arab Republic	Asia	Western Asia	Developing regions	315	419	409	269	264	385
Tajikistan	Asia	Central Asia	Developing regions	0	0	0	0	0	0
Thailand	Asia	South-Eastern Asia	Developing regions	56	53	113	65	82	66
Turkey	Asia	Western Asia	Developing regions	481	874	706	280	338	202
Turkmenistan	Asia	Central Asia	Developing regions	0	0	0	0	0	0
United Arab Emirates	Asia	Western Asia	Developing regions	0	2	2	1	2	0
Uzbekistan	Asia	Central Asia	Developing regions	0	0	0	0	0	0
Viet Nam	Asia	South-Eastern Asia	Developing regions	1191	1829	2162	3404	7583	5907
Yemen	Asia	Western Asia	Developing regions	1	2	1	6	0	15

49 rows x 38 columns

```
In [47]: # we can pass multiple criteria in the same line.
# let's filter for AreaName = Asia and RegName = Southern Asia

df_can[(df_can['Continent']=='Asia') & (df_can['Region']=='Southern Asia')]

# note: When using 'and' and 'or' operators, pandas requires we use '&' and
# don't forget to enclose the two conditions in parentheses
```

Out [47]:

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985
Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340
Bangladesh	Asia	Southern Asia	Developing regions	83	84	86	81	98	92
Bhutan	Asia	Southern Asia	Developing regions	0	0	0	0	1	0
India	Asia	Southern Asia	Developing regions	8880	8670	8147	7338	5704	4211
Iran (Islamic Republic of)	Asia	Southern Asia	Developing regions	1172	1429	1822	1592	1977	1648
Maldives	Asia	Southern Asia	Developing regions	0	0	0	1	0	0
Nepal	Asia	Southern Asia	Developing regions	1	1	6	1	2	4
Pakistan	Asia	Southern Asia	Developing regions	978	972	1201	900	668	514
Sri Lanka	Asia	Southern Asia	Developing regions	185	371	290	197	1086	845

9 rows × 38 columns

Exercise: Fetch the data where AreaName is 'Africa' and RegName is 'Southern Africa'. Display the dataframe and find out how many instances are there?

In [49]: `df_can[(df_can['Continent']=='Africa') & (df_can['Region']=='Southern Africa`

Out [49]:

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	19
Botswana	Africa	Southern Africa	Developing regions	10	1	3	3	7	4	
Lesotho	Africa	Southern Africa	Developing regions	1	1	1	2	7	5	
Namibia	Africa	Southern Africa	Developing regions	0	5	5	3	2	1	
South Africa	Africa	Southern Africa	Developing regions	1026	1118	781	379	271	310	
Swaziland	Africa	Southern Africa	Developing regions	4	1	1	0	10	7	

5 rows × 38 columns

► [Click here for a sample python solution](#)

Sorting Values of a Dataframe or Series

You can use the `sort_values()` function is used to sort a DataFrame or a Series based on one or more columns.

You to specify the column(s) by which you want to sort and the order (ascending or descending). Below is the syntax to use it:-

```
df.sort_values(col_name, axis=0, ascending=True, inplace=False, ignore_index=False)
```

`col_name` - the column(s) to sort by.

`axis` - axis along which to sort. 0 for sorting by rows (default) and 1 for sorting by columns.

`ascending` - to sort in ascending order (True, default) or descending order (False).

`inplace` - to perform the sorting operation in-place (True) or return a sorted copy (False, default).

`ignore_index` - to reset the index after sorting (True) or keep the original index values (False, default).

Let's sort out dataframe `df_can` on 'Total' column, in descending order to find out the top 5 countries that contributed the most to immigration to Canada.

```
In [50]: df_can.sort_values(by='Total', ascending=False, axis=0, inplace=True)
top_5 = df_can.head(5)
top_5
```

```
Out [50]:
```

	Continent	Region	DevName	1980	1981	1982	1983	1984	198
India	Asia	Southern Asia	Developing regions	8880	8670	8147	7338	5704	421
China	Asia	Eastern Asia	Developing regions	5123	6682	3308	1863	1527	181
United Kingdom of Great Britain and Northern Ireland	Europe	Northern Europe	Developed regions	22045	24796	20620	10015	10170	956
Philippines	Asia	South-Eastern Asia	Developing regions	6051	5921	5249	4562	3801	315
Pakistan	Asia	Southern Asia	Developing regions	978	972	1201	900	668	51

5 rows x 38 columns

Exercise: Find out top 3 countries that contributes the most to immigration to Canada in the year 2010.

Display the country names with the immigrant count in this year

```
In [52]: df_can.sort_values(by='2010', ascending=False, axis=0, inplace=True)
top_3 = df_can['2010'].head(3)
top_3
```

```
Out[52]: Philippines    38617
India                34235
China                30391
Name: 2010, dtype: int64
```

► [Click here for a sample python solution](#)

Congratulations! you have learned how to wrangle data with Pandas. You will be using a lot of these commands to preprocess the data before it can be used for data visualization.

Thank you for completing this lab!

Author

[Alex Aklson](#)

Other Contributors

[Jay Rajasekharan](#), [Ehsan M. Kermani](#), [Slobodan Markovic](#), [Weiqing Wang](#), [Dr. Pooja](#)

<!-- --!>

Change Log

| Date (YYYY-MM-DD) | Version | Changed By | Change Description |
|-------------------|---------|---------------|------------------------------------|
| 2023-06-08 | 2.5 | Dr. Pooja | Separated from original lab |
| 2021-05-29 | 2.4 | Weiqing Wang | Fixed typos and code smells. |
| 2021-01-20 | 2.3 | Lakshmi Holla | Changed TOC cell markdown |
| 2020-11-20 | 2.2 | Lakshmi Holla | Changed IBM box URL |
| 2020-11-03 | 2.1 | Lakshmi Holla | Changed URL and info method |
| 2020-08-27 | 2.0 | Lavanya | Moved Lab to course repo in GitLab |

-
-!>

© IBM Corporation 2020. All rights reserved.

In []: